COMPSCI753 Assignment 1: Graph Mining

Hugo Phibbs (UPI: hph433, ID: 533259260)

July 2024

1 Task 1

1.1 (A) Implementing a basic PageRank algorithm

Please see the Jupyter Notebook.

1.2 (B) Basic PageRank results

The results for this task are as below:

1. The algorithm's running time was **0.71 seconds**
2. The number of iterations to converge was **49 iterations**
3. For the scores and IDs of the top 10 ranked nodes, please see Table 1 below

Node ID	Node Score
41909	2.41e-04
1536	2.49e-04
791675	2.95e-04
865192	3.20e-04
577518	3.60e-04
779617	4.28e-04
370344	5.14e-04
544138	5.14e-04
24576	5.14e-04
747106	5.30e-04

Table 1: IDs and scores of the top 10 ranked nodes for Task 1

2 Task 2 Understanding dead-ends

2.1 (A) Finding dead-end nodes

I found the number of dead-end nodes (the number of nodes with no outgoing links) to be **187880**

2.2 (B) Calculating and Plotting leaked PageRank scores

The leaked page rank score per iteration can be calculated by finding the difference between the total rank score in a given iteration, and 1 (we expect the sum of rank values across nodes to equal to 1).

Figure 1 shows the change in leaked page rank scores per iteration using the algorithm implemented for Task 1. Figure 2 shows the total page rank per iteration. Together these can give a good picture of how the system's total PageRank score changes.

The general phenomenon is that the total PageRank score progressively leaks through the dead-end nodes, resulting in fewer total PageRank for the nodes that aren't dead-ends. The rate of PageRank leakage is initially quick before slowly levelling out - indicating some sort of asymptotic behaviour. This asymptotic behaviour is not surprising - given that the underlying phenomenon is that rank scores are 'leaking out' at dead-end nodes, with nowhere for it to go due to lack of outbound links. Consequently this results in increasingly less and less score available to the entire system. The end behaviour is that the total rank score gradually decreases, and the leakage rate also slows down also.

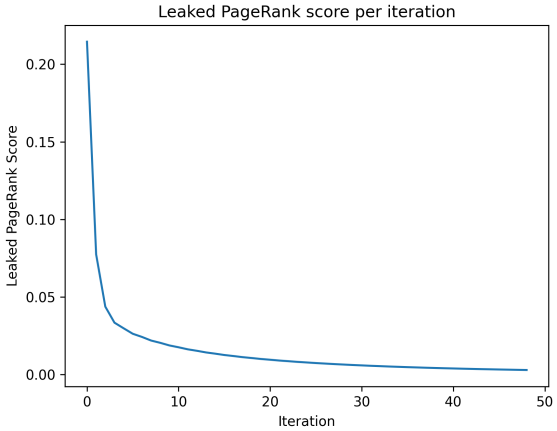


Figure 1: The leak in PageRank scores per iteration

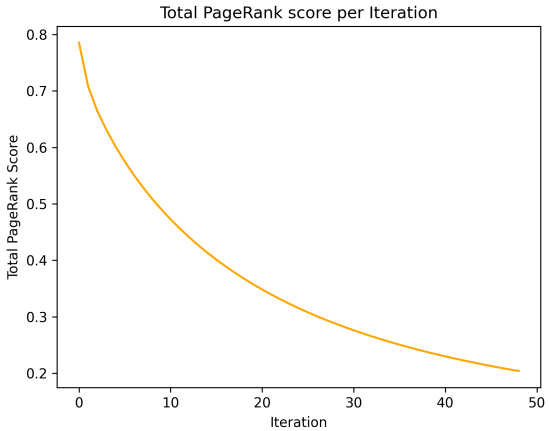


Figure 2: The leak in PageRank scores per iteration

3 Task 3

3.1 (A) Implementing a PageRank algorithm accounting for dead-ends and spider-traps

Please see the Jupyter Notebook for this task.

3.2 (B) Improved PageRank results

1. Algorithm running time was **0.15 seconds**
2. The number of iterations to converge was **12 iterations**
3. For the scores and IDs of the top 10 ranked nodes, please see Table 2 below

Node ID	Node Score
605856	7.74e-04
32163	7.87e-04
558791	8.15e-04
486980	8.24e-04
504140	8.46e-04
384666	8.56e-04
163075	9.92e-04
537039	1.00e-03
597621	1.01e-03
41909	1.04e-03

Table 2: IDs and scores of the top 10 ranked nodes for Task 3

3.3 (C) Varying the teleportation probability

Table 2 shows the convergence rate using different values of β (the teleportation probability) for the PageRank implementation from Task 3.

Teleportation Probability (β)	Iterations to Converge
1.0	63
0.9	10
0.8	7
0.7	5
0.6	5

Table 3: Different teleportation probabilities and their iterations to converge

A general finding from this experiment is that increasing β generally increases the required number of iterations to converge. This may be explained by the idea that a high teleportation factor means that pages that would normally be highly ranked (naturally due to in and out-links) are now subdued due to a simulated effect of a user visiting any page on the web more likely. Thus, it takes harder for these normally higher-ranked pages to distinguish themselves from other web pages - resulting in a slower rate of convergence.